



*Review programma di Previsione Fabbisogno Energetico
(GitHub Copilot / GPT_4o)*

Review programma di Previsione Fabbisogno Energetico (GitHub Copilot / GPT_4o)

1.Introduzione

Dopo aver descritto nel precedente articolo l'esperienza di collaborazione con GitHub Copilot all'interno di VSCode, il passo successivo è stato provare a capire con maggiore profondità cosa fosse realmente cambiato nel programma sottoposto a revisione.

Il miglioramento del codice era evidente già a una prima esecuzione: la nuova versione risultava più stabile, più leggibile e apparentemente anche più robusta. Tuttavia, nel lavoro con modelli di previsione basati su serie temporali — come accade frequentemente nel settore Energy & Utilities — l'impressione iniziale non è mai sufficiente. Piccole modifiche nella pipeline possono avere effetti significativi sulla qualità metodologica del modello.

Per questo motivo **ho deciso di effettuare un'analisi comparata sistematica tra le due versioni del notebook.**

La prima versione, denominata **"Previsione Fabbisogno v04"**, rappresentava il programma originale sviluppato per realizzare il modello di previsione dei consumi elettrici.

La seconda versione, **"Previsione Fabbisogno (modificato) v06"**, era invece il risultato di una serie di interventi progressivi realizzati con il supporto di strumenti di intelligenza artificiale, in particolare GitHub Copilot e successivamente GPT-5.3.

In quest'articolo illustrerò quest'attività ed i risultati ottenuti.

2. L'analisi comparata dei notebook: cosa è cambiato davvero nel codice

L'obiettivo dell'analisi non era quello di modificare ulteriormente il codice (almeno non nell'immediato), ma di comprenderne la struttura metodologica e individuare le differenze tra le due implementazioni. In altre parole, l'idea era quella di realizzare una sorta di **audit tecnico** della pipeline di forecasting.

Per rendere il confronto il più possibile rigoroso, l'analisi è stata effettuata sull'intero notebook, esaminando sia le celle di codice sia quelle descrittive. L'attenzione non si è concentrata su singole funzioni isolate, ma sull'intera pipeline di lavoro: dal caricamento dei dati fino alla generazione delle previsioni.

Questo approccio è particolarmente importante quando si lavora con modelli di previsione delle serie temporali, perché eventuali errori metodologici non si trovano necessariamente nel modello in sé, ma spesso nella preparazione dei dati, nella validazione o nella costruzione delle feature.

L'analisi ha quindi considerato in modo sistematico tutte le fasi della pipeline: caricamento dei dati, preprocessing, feature engineering, addestramento del modello, validazione, valutazione delle prestazioni e visualizzazione dei risultati.

Da questo confronto sono emerse numerose differenze tra le due versioni del notebook. Alcune sono modifiche relativamente piccole ma utili per rendere il codice più robusto. Altre invece rappresentano cambiamenti più significativi nella progettazione della pipeline o nella configurazione del modello.

Nel complesso, le modifiche possono essere raggruppate in alcune grandi categorie: miglioramenti di robustezza del codice, evoluzione dell'architettura del modello, arricchimento delle feature, maggiore controllo metodologico sulla pipeline e introduzione di strumenti diagnostici più evoluti.

Vediamole nel dettaglio.

Miglioramenti di robustezza nella gestione dei dati

Una delle prime differenze che emergono nel confronto tra i due notebook riguarda l'introduzione di controlli aggiuntivi sulla struttura dei dati.

In particolare, nella versione più recente sono stati inseriti alcuni controlli espliciti sul tipo di indice temporale utilizzato nei DataFrame. Questo può sembrare un dettaglio marginale, ma in realtà rappresenta una pratica molto importante nella programmazione di pipeline di data processing.

Nel caso delle serie temporali, infatti, molte operazioni — come l'interpolazione temporale o la generazione di nuove feature basate sull'orario — presuppongono che l'indice del dataset sia un oggetto di tipo **DatetimeIndex**. Se l'indice non ha questa struttura, alcune operazioni possono produrre risultati errati oppure generare errori difficili da interpretare.

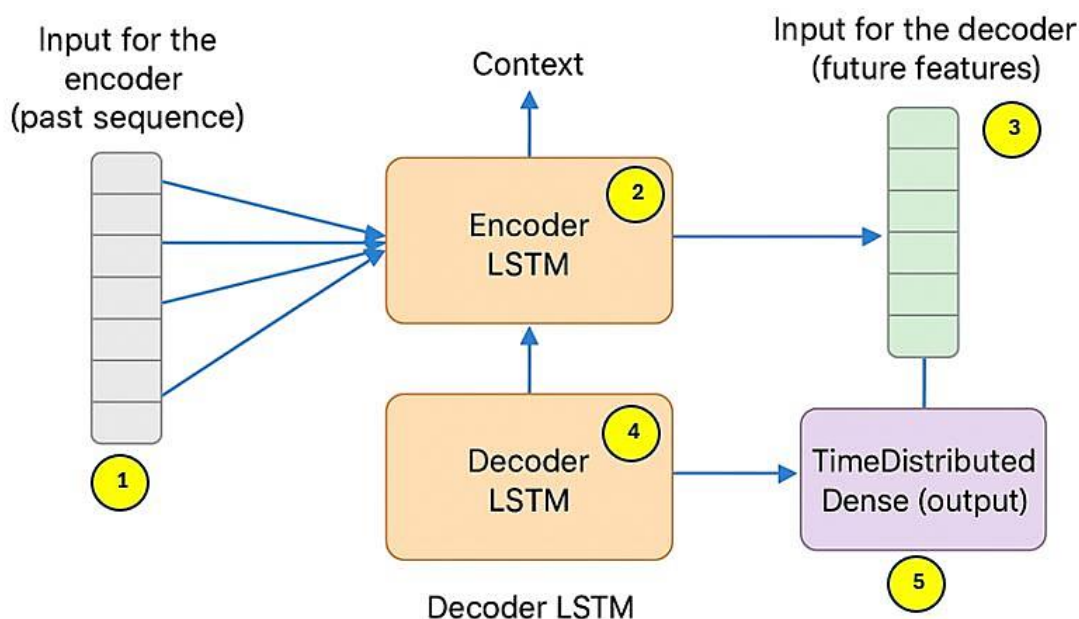
Per evitare questo tipo di problemi, nella nuova versione del codice è stato introdotto un controllo preliminare che verifica esplicitamente il tipo dell'indice. Se l'indice non è di tipo temporale, l'esecuzione viene interrotta e viene restituito un messaggio di errore chiaro.

Si tratta di una tipica forma di **programmazione difensiva**, molto diffusa nei progetti di data science destinati a evolvere nel tempo o a essere riutilizzati in contesti diversi.

Dal punto di vista funzionale, queste modifiche non cambiano il comportamento del codice quando i dati sono corretti. Tuttavia aumentano significativamente l'affidabilità del programma quando viene utilizzato in pipeline più complesse o con dataset differenti.

Evoluzione dell'architettura del modello

Un secondo gruppo di modifiche riguarda invece la configurazione del modello di machine learning utilizzato per la previsione.



Il modello impiegato nel notebook è basato su una rete neurale LSTM seq2seq ¹, una tipologia di architettura particolarmente adatta alla modellazione delle serie temporali perché consente di catturare dipendenze nel tempo anche su orizzonti relativamente lunghi.

Nella nuova versione del codice sono stati modificati alcuni parametri chiave del modello.

In primo luogo è stata aumentata la dimensione dello spazio latente della rete, cioè il numero di unità interne utilizzate dall'LSTM per rappresentare le informazioni temporali. Questa modifica consente al modello di avere una maggiore capacità rappresentativa e quindi di catturare pattern più complessi presenti nei dati.

Un secondo cambiamento riguarda la gestione dell'orizzonte di previsione. Nella versione originale questo valore era definito direttamente nel codice come costante numerica. Nella versione aggiornata invece è stato trasformato in una variabile configurabile centralizzata.

Questo tipo di scelta rende il codice più flessibile e più facile da mantenere, perché consente di modificare l'orizzonte di previsione senza intervenire manualmente in più punti del programma.

Un ulteriore intervento ha riguardato il parametro di **dropout**, che controlla il livello di regolarizzazione della rete neurale. Una riduzione moderata di questo parametro può

¹ I modelli Seq2Seq sono [descritti in dettaglio nelle note finali](#)

permettere al modello di apprendere meglio le dinamiche dei dati mantenendo comunque una certa protezione dall'overfitting.

Infine è stato introdotto un controllo esplicito sul **learning rate** dell'ottimizzatore utilizzato durante l'addestramento del modello. Questo consente di avere un maggiore controllo sul processo di convergenza dell'algoritmo.

Nel complesso queste modifiche non cambiano l'architettura generale del modello, ma ne migliorano la configurabilità e la stabilità durante l'addestramento.

Maggiore flessibilità nella gestione dei dati storici

Un altro cambiamento interessante riguarda la possibilità di scegliere quale intervallo temporale utilizzare come storico per l'addestramento del modello.

Nella versione originale del notebook, l'indice temporale veniva sempre generato in modo automatico tramite una singola funzione. La versione modificata introduce invece la possibilità per l'utente di selezionare tra diverse modalità di generazione dello storico.

Questa modifica rende la pipeline più flessibile e consente di utilizzare lo stesso codice sia per analisi su dataset reali sia per simulazioni o test.

In pratica il programma può ora essere utilizzato in due modalità operative: una modalità basata su dati storici reali già disponibili e salvati in precedenza su disco e una modalità più dinamica utile per generare dataset sintetici o effettuare simulazioni.

Si tratta di una modifica apparentemente semplice ma che aumenta la versatilità dello script e lo rende più adatto a diversi scenari di utilizzo.

Miglioramento dell'organizzazione dei file

Un'altra area di intervento riguarda la gestione dei percorsi dei file utilizzati dal programma. Nella versione originale alcuni file venivano salvati utilizzando percorsi assoluti specifici del sistema operativo Windows. Questo approccio può funzionare in un contesto locale, ma riduce notevolmente la portabilità del codice.

Nella versione aggiornata è stata introdotta una gestione più strutturata dei percorsi utilizzando la libreria **pathlib**, che consente di costruire percorsi compatibili con diversi sistemi operativi.

Inoltre il codice ora crea automaticamente la cartella destinata a contenere i dati nel caso in cui non esista già. Questo evita errori durante il salvataggio dei file e migliora l'organizzazione complessiva del progetto.

Anche in questo caso si tratta di modifiche che non cambiano la logica del modello ma rendono il codice più professionale e più facile da utilizzare in contesti diversi.

Arricchimento delle feature temporali

Uno dei cambiamenti più rilevanti riguarda il **feature engineering**, cioè il processo di costruzione delle variabili utilizzate dal modello.

Nel settore dell'energia, i consumi elettrici presentano pattern temporali molto marcati. Esistono cicli giornalieri legati alle attività umane, differenze tra giorni feriali e weekend, e stagionalità legate al periodo dell'anno.

Per permettere al modello di catturare queste dinamiche, nella nuova versione del notebook sono state introdotte alcune nuove feature temporali.

Tra queste troviamo **l'ora del giorno** e il **giorno della settimana**, oltre a una codifica ciclica di queste variabili mediante funzioni seno e coseno. Questo tipo di rappresentazione è molto utile perché evita discontinuità artificiali tra valori consecutivi, come nel passaggio da mezzanotte all'una o dalla domenica al lunedì.

Inoltre è stata introdotta una **feature autoregressiva**, cioè una variabile che rappresenta il consumo registrato nel passo temporale precedente. Nelle serie temporali energetiche questa informazione è spesso estremamente informativa, perché il consumo tende a essere fortemente correlato con i valori immediatamente precedenti.

Nel complesso, queste nuove feature rendono il dataset più ricco e permettono al modello di apprendere pattern temporali più complessi.

Maggiore controllo metodologico sulla pipeline

Un'altra differenza significativa riguarda l'introduzione di controlli metodologici più rigorosi nella gestione della pipeline di addestramento.

Uno dei rischi più comuni nei modelli di serie temporali è il cosiddetto **data leakage**, cioè la presenza involontaria di informazioni future all'interno del processo di addestramento del modello.

Per ridurre questo rischio, nella versione aggiornata del codice è stata migliorata la separazione tra dataset di training, validazione e test.

Inoltre sono stati introdotti controlli automatici per verificare che gli intervalli temporali dei diversi dataset non si sovrappongano. Questo tipo di verifica è fondamentale per garantire che il modello venga valutato su dati realmente fuori campione.

Un ulteriore miglioramento riguarda la gestione dello **scaling dei dati**, che ora viene effettuato utilizzando parametri calcolati esclusivamente sul dataset di training. Questa pratica è essenziale per evitare contaminazioni tra training e test set.

Miglioramenti nella fase di addestramento del modello

La nuova versione del notebook introduce anche alcune modifiche nella fase di training del modello.

Una di queste riguarda l'utilizzo di un dataset di validazione esplicito durante l'addestramento, invece di affidarsi alla suddivisione casuale automatica dei dati.

Questo approccio è generalmente preferibile nel caso delle serie temporali, perché consente di mantenere l'ordine cronologico dei dati.

Inoltre il codice ora salva automaticamente il modello addestrato e lo storico del processo di training. Questo permette di analizzare successivamente l'andamento dell'addestramento senza dover ripetere l'intero processo.

Introduzione di strumenti diagnostici

Una delle evoluzioni più interessanti riguarda l'introduzione di una sezione dedicata all'analisi diagnostica delle prestazioni del modello.

La nuova versione del notebook include infatti alcune funzioni che consentono di visualizzare graficamente il confronto tra valori reali e valori previsti dal modello.

Tra queste troviamo grafici scatter che mostrano la relazione tra osservazioni e previsioni, grafici temporali che confrontano l'andamento delle due serie e grafici che rappresentano l'errore di previsione nel tempo.

Questi strumenti sono molto utili perché permettono di individuare rapidamente eventuali bias sistematici del modello o pattern di errore che potrebbero suggerire ulteriori miglioramenti.

Miglioramenti nella logica di forecasting

Infine, alcune modifiche riguardano la fase di previsione vera e propria.

In particolare è stata migliorata la gestione dello scaling dei dati utilizzati per le previsioni future, garantendo maggiore coerenza tra la fase di addestramento e quella di inferenza.

È stato inoltre introdotto un sistema di logging più strutturato che consente di monitorare l'esecuzione del programma e diagnosticare eventuali problemi.

Un altro miglioramento riguarda il calcolo automatico di alcune metriche di valutazione delle previsioni, come MAE, RMSE e MAPE, che permettono di avere immediatamente una misura quantitativa della qualità delle previsioni generate.

Una valutazione complessiva delle modifiche

Nel complesso, il passaggio dalla versione v04 alla versione v06 del notebook rappresenta un miglioramento significativo della pipeline di forecasting.

Le modifiche introdotte migliorano la robustezza del codice, arricchiscono il dataset con nuove informazioni utili per il modello e introducono controlli metodologici più rigorosi.

Allo stesso tempo, l'analisi evidenzia anche alcuni aspetti che richiedono attenzione, in particolare per quanto riguarda il rischio di data leakage e la corretta validazione delle serie temporali.

Si tratta tuttavia di considerazioni tipiche di qualunque progetto di data science applicato a sistemi reali e non riducono il valore complessivo delle migliorie introdotte.

Piuttosto, evidenziano come il lavoro con strumenti di intelligenza artificiale — se utilizzato in modo consapevole — possa contribuire a migliorare significativamente la qualità del codice e la maturità metodologica di un progetto.

3. Conclusioni: verso una nuova collaborazione tra sviluppatori e intelligenza artificiale

L'esperienza descritta in questo studio rappresenta, nel suo piccolo, un esempio concreto di come stia cambiando il modo di sviluppare software e modelli di analisi dei dati. Nel caso specifico, il punto di partenza era un notebook Python dedicato alla previsione dei consumi elettrici, sviluppato in ambito Energy & Utilities. Un tipico progetto di data science applicata: acquisizione dei dati, preprocessing, costruzione delle feature, addestramento del modello e valutazione delle prestazioni.

Ciò che rende interessante questa esperienza non è tanto il modello utilizzato — una rete LSTM/Seq2Seq per il forecasting delle serie temporali — quanto il processo di sviluppo che ha portato alla sua evoluzione. Il codice non è stato migliorato esclusivamente attraverso un lavoro manuale tradizionale, ma attraverso una forma di collaborazione tra sviluppatore umano e strumenti di intelligenza artificiale integrati nell'ambiente di sviluppo.

Da un lato, strumenti come **GitHub Copilot** hanno consentito di analizzare il codice esistente e suggerire modifiche puntuali orientate alla qualità del software: maggiore robustezza nella gestione dei dati, migliore organizzazione della pipeline, controlli

metodologici più espliciti. Dall'altro lato, strumenti conversazionali come **ChatGPT** hanno permesso di discutere, interpretare e contestualizzare queste modifiche, trasformando una semplice revisione del codice in un vero e proprio esercizio di audit metodologico.

Il risultato non è stato semplicemente un codice “più pulito”. La nuova versione del notebook ha mostrato un miglioramento complessivo sotto diversi aspetti: maggiore solidità nella gestione dei dati, maggiore chiarezza nella pipeline di addestramento, un arricchimento delle feature utilizzate dal modello e una migliore capacità di analizzare le prestazioni attraverso strumenti diagnostici più evoluti.

Ma forse il risultato più interessante riguarda il metodo di lavoro.

Tradizionalmente, l'evoluzione di un progetto di data science segue un processo abbastanza lineare: lo sviluppatore scrive il codice, lo testa, lo modifica, e progressivamente lo migliora attraverso iterazioni successive. Con l'introduzione di strumenti di assistenza basati su intelligenza artificiale, questo processo si trasforma. Il codice diventa sempre più il risultato di un dialogo continuo tra l'esperienza del programmatore, la conoscenza del dominio applicativo e la capacità degli strumenti AI di individuare pattern, suggerire soluzioni e individuare potenziali criticità.

In questo scenario emergono nuove forme di collaborazione. Il data scientist o lo sviluppatore non viene sostituito dall'intelligenza artificiale, ma assume sempre più il ruolo di **orchestratore del processo**, colui che guida l'analisi, verifica le ipotesi, valuta la correttezza metodologica delle soluzioni proposte e decide quali suggerimenti integrare realmente nel progetto.

Gli strumenti AI diventano quindi una sorta di “**copilota cognitivo**”, capace di accelerare alcune attività – come la revisione del codice, l'individuazione di errori o la generazione di strutture di pipeline – lasciando però all'essere umano il compito fondamentale di interpretare il contesto applicativo e garantire la correttezza scientifica del modello.

Nel settore **Energy & Utilities**, dove i modelli di previsione vengono spesso utilizzati per supportare decisioni operative e pianificazione energetica, questo aspetto è particolarmente importante. Un algoritmo può essere tecnicamente sofisticato, ma se non è costruito con una pipeline metodologicamente corretta rischia di produrre risultati fuorvianti. Proprio per questo la collaborazione tra competenze umane e strumenti intelligenti può diventare un elemento di grande valore.

Guardando al futuro, è plausibile che questo tipo di interazione diventi sempre più naturale negli ambienti di sviluppo. Gli IDE integreranno in modo crescente agenti intelligenti capaci non solo di suggerire codice, ma anche di analizzare intere architetture software, verificare pipeline di machine learning, individuare potenziali problemi metodologici e proporre strategie di miglioramento.

In altre parole, potremmo assistere alla nascita di **ecosistemi di sviluppo ibridi**, in cui programmatori, data scientist e agenti AI lavorano insieme all'interno dello stesso ambiente operativo ².

In questo contesto il valore dell'esperienza umana non diminuisce, ma cambia natura. Diventa sempre meno legato alla scrittura manuale di ogni singola riga di codice e sempre più legato alla capacità di progettare sistemi complessi, interpretare i risultati e garantire la qualità metodologica delle soluzioni sviluppate.

L'esperienza descritta in questo articolo rappresenta quindi un piccolo laboratorio di questo possibile futuro: un esempio concreto di come strumenti di intelligenza artificiale possano affiancare il lavoro dei professionisti del software e della data science, contribuendo a rendere i progetti più robusti, più trasparenti e più maturi dal punto di vista metodologico.

E probabilmente siamo solo all'inizio di questo percorso.

4. Note Tecniche

◆ Reti LSTM seq2seq

Una **rete neurale LSTM seq2seq (sequence-to-sequence)** è un'architettura di deep learning progettata per **modellare e prevedere sequenze temporali**, risultando particolarmente efficace nel forecasting di grandezze dinamiche come il **fabbisogno energetico**.

Nel contesto di questa applicazione, la rete utilizza una struttura **encoder-decoder basata su LSTM (Long Short-Term Memory)**. Le LSTM sono una variante delle reti neurali ricorrenti (RNN) progettate per **apprendere dipendenze temporali anche molto lunghe**, grazie a meccanismi interni di memoria che consentono di mantenere informazioni rilevanti nel tempo e di evitare problemi tipici delle RNN classiche come il *vanishing gradient*.

L'architettura **seq2seq** è composta da due parti principali. L'**encoder** riceve in ingresso una sequenza di dati storici (ad esempio consumi passati, variabili meteo e feature temporali) e la comprime in una rappresentazione interna chiamata **stato latente**, che sintetizza il comportamento recente del sistema energetico. Questo stato latente rappresenta una sorta di "memoria" del contesto temporale.

Il **decoder** utilizza questo stato latente come punto di partenza per generare una nuova sequenza di output, cioè **le previsioni dei consumi energetici per l'orizzonte**

² Per un maggior dettaglio [consultare le note in coda al documento](#)

futuro (ad esempio le prossime ore o i prossimi giorni). In questo processo il decoder può anche utilizzare informazioni future disponibili, come **previsioni meteo**, che influenzano significativamente il fabbisogno energetico.

Il modello quindi apprende una funzione che mappa:

- **una sequenza di input storici** (consumi passati, meteo, variabili temporali)
- **in una sequenza di output futuri** (previsioni di consumo energetico).

Questo approccio è particolarmente adatto alle **serie temporali energetiche** perché riesce a catturare:

- pattern **giornalieri e settimanali** dei consumi,
- dipendenze tra **condizioni meteo e domanda energetica**,
- dinamiche temporali **non lineari e complesse**.

In sintesi, la rete LSTM seq2seq utilizzata in questa applicazione consente di **trasformare una sequenza di dati storici e variabili esogene in una sequenza di previsioni future**, fornendo uno strumento efficace per stimare il fabbisogno energetico e supportare attività di pianificazione e gestione dei sistemi energetici.

◆ Integrazione con Agenti AI

Nell'ultimo periodo sono stati annunciati un paio di prodotti che in modalità diverse espletano il ruolo di Agenti AI. In particolare:

1. Funzione Codex nella barra laterale di ChatGPT

Quella che si vede nel cruscotto di ChatGPT (sulla sinistra) è **Codex integrato dentro l'interfaccia di ChatGPT sul web / app**. Si comporta come un **assistente di coding o agente intelligente** che si può invocare con prompt per:

- scrivere o migliorare codice,
- fare refactor,
- generare snippet,
- rispondere a domande su progetti,
- analizzare repository ecc.

Questa esperienza è **incorporata nell'interfaccia conversazionale** e fa parte del piano ChatGPT (es. Plus, Pro, Business, ecc.).ed è pensata per un uso **interattivo**, senza uscire dalla finestra di ChatGPT: digiti istruzioni, ottieni risultati e puoi copiare/incollare nel tuo editor.

2. App desktop ufficiale Codex (Windows / Microsoft Store)

La nuova **app Codex** disponibile in Microsoft Store è un **programma separato** rispetto a ChatGPT, progettato per sviluppatori e team che vogliono **gestire agenti Codex su progetti reali**:

- è una **interfaccia dedicata**, non solo un'estensione della chat;
- permette di **orchestrare più agenti in parallelo** su più attività o repository;
- include **strumenti avanzati** come supporto per worktree git, visualizzazione diff, commenti sulle modifiche, automazioni e competenze (skills);
- supporta **task di lunga durata**, non solo richieste singole nella stessa chat;
- può essere visto come un **centro di comando** per lavorare con Codex (in locale o delegando nel cloud) su flussi di sviluppo complessi;
- ha una **sandbox di sicurezza nativa**, separata dall'ambiente standard ChatGPT, per proteggere il tuo sistema.

Quindi **non è la stessa cosa** che l'icona o il pulsante di Codex dentro ChatGPT: quella è semplicemente un modo comodo di usare un agente all'interno della chat, mentre l'app desktop è un **ambiente di lavoro completo per sviluppatori**, molto più potente e integrato con i workflow reali di progetto.

In breve

- **Codex nel cruscotto di ChatGPT**: lo usi come "assistente di coding" direttamente nella chat — ottimo per richieste puntuali, snippet, domande e correzioni rapide.
- **App desktop Codex**: è un **client standalone** pensato per flussi di sviluppo veri e propri, con gestione avanzata di agenti, workspace e automazioni.

Se usi Codex *solo per singole richieste di aiuto sul codice*, la versione nel cruscotto è sufficiente. Se invece vuoi lavorare su progetti più complessi, coordinare più agenti e mantenere il contesto tra molte attività lungo tempo, **l'app desktop è la soluzione più completa**.

4. Appendice

Allego al presente documento i due notebook jupyter contenenti i programmi python (originale e quello modificato):

- 📌 Notebook originale : [Previsione Fabbisogno v04](#)
- 📌 Notebook modificato : [Previsione Fabbisogno \(modificato\) v06](#)
- 📌 il [dettaglio tecnico della review](#) operata sul notebook modificato

◆ Per un maggior dettaglio dello studio fatto sull'utilizzo di Codex di OpenAI e GitHub Copilot :



Codice sotto la lente: la mia
esperienza con Codex di OpenAI